MxBoost: Mutual-exclusive Boosting for Online Classification

Huaxin You HXYOU@PSTAT.UCSB.EDU
Department of Statistics
University of California, Santa Barbara
Santa Barbara, CA 93106

Edward Chang ECHANG@ECE.UCSB.EDU
Department of Electrical & Computer Engineering
University of California, Santa Barbara
Santa Barbara, CA 93106

Abstract

In this paper we discuss how to improve classifier ensembles for online classification problems where classification has to be performed quickly. Under the restriction on time and therefore ensemble size, we propose *MxBoost (Mutual-exclusive Boosting)* based on the heuristic that members of a decision committee should make mistakes mutually exclusively. Theoretical justifications are given to explain the effectiveness of MxBoost. Furthermore, we conduct extensive empirical studies using representative ensemble schemes including One Per Class (OPC) and Error Correcting Output Coding (ECOC). Our experiments show that MxBoost improves the class-prediction accuracy of both OPC and ECOC on all standard and image datasets that we tested while maintaining good runtime performance. Our experiments also show that MxBoost does not suffer from the overfitting problem.

1. Introduction

Many emergent Internet applications,such as network-intruder detection, video surveillance, and email/Web-site filtering, demand efficient online classifiers. In this paper, we propose a Mutual-exclusive Boosting (MxBoost) algorithm to achieve high class-prediction accuracy, as well as to maintain fast K-class classification.

The K-class classification problem can be solved by employing the regression approach, which considers the discrimination problem of K classes simultaneously. However, Friedman (1996) points out that the regression approach can be computationally intensive $(O(K^3))$ and may impose a universal assumption on all class boundaries. Many recent studies have proposed ensemble schemes, which use binary (two-class) algorithms to solve K-class classification problems. Decomposing a K-class classification problem into a number of binary classification problems allows an ensemble scheme to model binary class boundaries with much greater flexibility at a lower computational cost (in the order of $O(K^2)$) than that of the regression approach.

To improve the prediction accuracy of an ensemble scheme, we propose MxBoost which forces ensemble members to make mutually exclusive mistakes. To explain how MxBoost works, we use two examples depicted in Figures 1 and 2 to answer the following two questions:

1. When is an ensemble optimal?

2. What does MxBoost differ from a traditional boosting scheme such as AdaBoost?

When Is an Ensemble Optimal?

Figure 1 depicts the prediction error of an ensemble of three subclassifers C_A, C_B, and C_C as region A, B, and C,

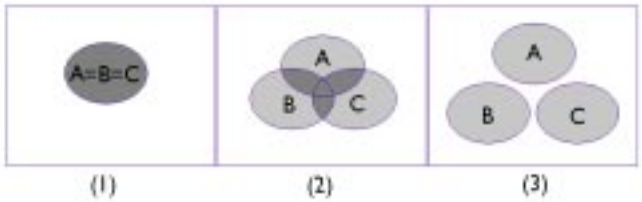


Figure 1. Dark regions are the portions of a population that are misclassified by the ensemble. Gray regions are the portions of the population that are misclassified by individual classifiers. Case (1): A, B, C are identical. Case (2): A, B, C are independent. Case (3): A, B, C are mutually exclusive.

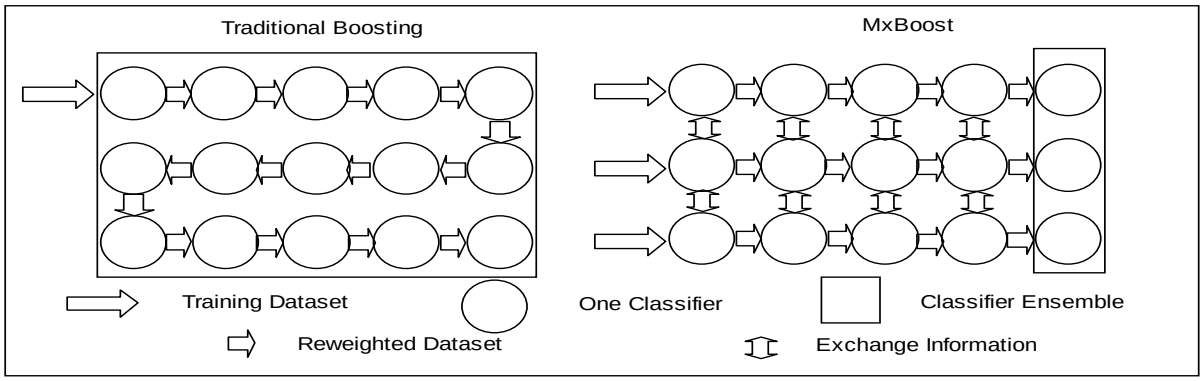


Figure 2. The difference between traditional Boosting and MxBoost

respectively. Suppose each classifier yields prediction errors of 25%. Figure 1(1) shows that when C_A, C_B, and C_C make identical errors, the ensemble has an error rate of 25%. Figure 1(2) shows that when C_A, C_B, and C_C make independent errors, the ensemble's error is reduced to $3 \times 0.25^2 - 3 \times 0.25^3 + 0.25^3 = 15.6\%$. When A, B, and C are mutually exclusive (i.e., no two classifiers make mistakes on the same instances), Figure 1(3) shows that the ensemble's prediction is perfect.

Remark 1: *An ensemble is optimal when all classifiers make mistakes mutually exclusively.*

The example depicted by Figure 1 reveals that having inter-classifier correlation may not be all bad. Especially when classifiers make mutually exclusive errors, the performance of their ensemble becomes optimal. In fact, Bagging (Breiman, 1998) can be considered as an ensemble depicted in Figure 1(2), which adds classifiers to an ensemble by training the base classifier to independent subsamples of the training set. Boosting (Friedman et al., 1998; Schapire et al., 1997) can be considered as an ensemble depicted in Figure 1(3), which adds new classifiers to an ensemble by reweighting the training instances in order that the next classifier will make mistakes on different instances (i.e., make mutually exclusive errors). MxBoost is a variant of Boosting. We explain below how MxBoost differs from the traditional boosting schemes.

What does MxBoost differ from AdaBoost?

A traditional boosting scheme such as AdaBoost adds classifiers sequentially to improve the prediction accuracy. As observed by Friedman et al. (1998) and Schapire et al. (1997), however, AdaBoost requires a large ensemble size to reduce error significantly. This large ensemble size translates not only to lengthy training time, but also to lengthy classification time. Figure 2 shows that a sequential boosting scheme typically requires a large number of classifiers to make an aggregated class prediction. (All classifiers in the rectangle participate in a class prediction.) This high cost of prediction cannot be tolerated by online classification applications in which a class-prediction must be performed quickly. MxBoost does not require a large number of ensemble members. As depicted in Figure 2, MxBoost usually involves only the last set of classifiers (shown in the rectangle) to make a class prediction. MxBoost improves class-prediction accuracy while keeping the prediction time fixed, independently of the number of boosting iterations.

In summary, the contributions of this paper are as follows:

1. We propose MxBoost to improve classifier ensembles by tuning classifiers to make mistakes mutually exclusively.

2. We study MxBoost on two representative ensembles: One Per Class (OPC) and Error Correcting Output Coding (ECOC). We demonstrate that MxBoost can improve class-prediction accuracy on both standard UCI datasets and a widely used image dataset. At the same time, since MxBoost keeps its ensemble size fixed, it performs classification efficiently.

3. We also show that MxBoost does not seem to suffer from overfitting via both theoretical analysis and empirical study.

The rest of this paper is organized into four sections. Section 2 presents MxBoost and explains why it works. Section 3 details the setup of our empirical study. Section 4 reports experimental results. Finally, we offer our concluding remarks in Section 5.

2. Improve Classifier Ensembles by MxBoost

In this section, we discuss in detail how classifiers can be tuned to make mistakes mutually exclusively. We first de-

fine *margin* and *mutually exclusive error*. Then we propose MxBoost to achieve mutual exclusiveness among ensemble members. Finally, we present theoretical justifications to explain why MxBoost works.

2.1 Margin and Mutually Exclusive Error

As in Schapire et al. (1997), we define the *margin* of an instance (i.e., an example, a data object) as the difference between the votes assigned to the correct class and the maximal votes assigned to any single incorrect class. It is assumed in this paper that a negative margin corresponds to a misclassification. We use two examples below to show the relationship between margin and mutually exclusive error.

Example 1: *For the case of binary classification, let a vote be either 0 or 1. An example's margin is simply the number of correct votes minus the number of wrong votes. Figure 3 illustrates how classifiers can be tuned to make errors mutually exclusively. It shows an ensemble of three classifiers of similar generalizing ability with respect to a two-class decision. As classifiers are tuned, the regions with positive margins expand, whereas the regions with negative margins are reduced. In other words, to achieve mutual exclusiveness among classifiers, we need to reduce the regions with negative margins. The regions with largest margins are also reduced for the reason stated in Remark 2.*

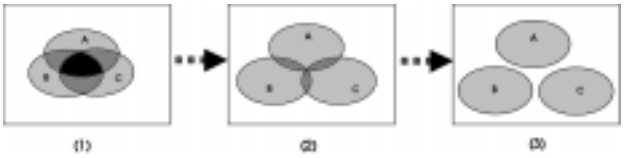


Figure 3. Dark regions show the portions of a population that have margin -3. Deep gray regions have margin -1. Light gray regions have margin 1. White regions have margin 3. Margins that are < 0 imply misclassifications. A, B, C are defined as in Figure 1. Two-class classification is assumed.

Remark 2: *For Example 1, MxBoost decreases the portion of examples with largest margins.*

Proof: Examples with largest positive margins are classified perfectly by all classifiers. From Figure 3, it is easy to see that the portion of examples with largest margins equals to

$$1 - P(A \cup B \cup C).$$

$P(.)$ is the probability measure. When A, B, C becomes more mutually exclusive, $1 - P(A \cup B \cup C)$ becomes smaller. Therefore, Remark 2 holds. □

Example 2: *For the case of K-class classification, OPC (One Per Class) ensemble scheme trains K binary classifiers, each of which separates one class from the other $(K-1)$ classes. Given a point x to classify, the binary classifier with the largest output determines the class of x. The voting function of OPC is:*

$$H(x) = V(h_1(x), h_2(x), ..., h_m(x)) = \textit{argmax}_i h_i(x),$$

where $h_i(x)$ outputs the confidence vote for class i. The margin for OPC is defined as:

$$\begin{aligned} & M(y(x), V(h_1(x), h_2(x), ..., h_m(x))) \\ = \quad & h_{i^*}(x) - \max_{i \neq i^*} h_i(x), \end{aligned}$$

where i^* is the true class for x.

As in Example 1, the regions with negative margins are considered as "overlapping" regions. To achieve mutual exclusiveness among OPC classifiers, we need to reduce the regions with negative margins. Since confidence voting is used in OPC, the definition of the largest margin is not evident. However, it can be observed from Section 4 that the examples with large margins are reduced both in size and magnitude. The same can be said of the traditional Boosting schemes (see Schapire et al. (1997), pages 13 – 14).

Section 2.2 provides a general algorithm that forces classifiers in an ensemble to make mutually exclusive errors. It can be applied for a variety of ensembles, regardless of the voting function in use or the margin calculation method.

2.2 MxBoost Learning Algorithm

To achieve the effect as shown in Figure 3, we propose to do the following. Suppose we have an ensemble that yields some training errors. For classifier h_i in the ensemble, we increase the weight for examples with negative margins (i.e., misclassified examples) according to their margins, and decrease the weight for examples with positive margins. After each iteration of retraining, classifier h_i is less likely to misclassify examples with negative margins. In this way, the new ensemble will more likely classify more examples into positive margins.

Let $h_1, h_2, ..., h_m$ be the classifiers in the ensemble. $V(h_1, h_2, ..., h_m)$ is the voting function in use. T is the number of iterations. The training set is $(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)$ where x_i's are feature vectors and y_i's are their labels, $i = 1, ..., n$. $M(y(x), V(h_1(x), h_2(x), ..., h_m(x)))$ is the margin for example x. β is a parameter that controls how aggressively the weights should be changed. The detailed algorithm is presented as follows.

MxBoost Learning Algorithm

1. Input $(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)$, β.

2. Initialize w_i^c to be $\frac{1}{n}$, $c = 1, ..., m$; $i = 1, ..., n$.

3. For $t = 1, ..., T$:

- For each $c = 1, ..., m$, train classifier functions h_c on (x_i, y_i) with weights w_i^c.

- For each c, adjust the weights w_i^c's as follows:

$$w_i^c(t+1) \leftarrow w_i^c(t)\exp(-\beta M(y(x_i), V(h_1(x_i), ..., h_{c-1}(x_i), h_{c+1}(x_i), ..., h_m(x_i)))). \quad (1)$$

- For each c, normalize w_i^c's so that they have sum one.

4. Output the final ensemble:

$$H(x) = V(h_1(x), h_2(x), ..., h_m(x))$$

To insure that A, B, C (in Figure 3) will remain approximately the same size after the training set is reweighted, we choose small values of β in practice. As shown in Section 4.5, if β is big, MxBoost may fail to improve the ensemble.

We call this method MxBoost since intuitively MxBoost forces classifiers to make mutually exclusive mistakes. Similar to traditional Boosting, MxBoost tries to boost the margins (to make more of them positive) and shares the side-effect stated in Remark 2. Unlike traditional Boosting, MxBoost does not add classifiers into an ensemble. Both theoretical results in Section 2.3 and empirical results in Section 4 show that MxBoost is an effective way to improve classifier ensembles.

2.3 Theoretical Justifications

This section explains when and why MxBoost works. The main result is stated in Remark 5. In proving Remark 5, we present and utilize Theorem 1 which is a restatement of Theorem 1 in Schapire et al. (1997). Let us first define a few terms.

Definition 1. For different choices of M, let ψ be the value such that $M > \psi$ implies a correct classification, $M < \psi$ implies a wrong classification, and $M = \psi$ corresponds to a tied decision. In Schapire et al. (1997) , $\psi = 0$, whereas in Friedman et al. (1998) $\psi = 1$.

Definition 2. Let S, D and A denote the original training margins, population margins, and the training margins after MxBoost. Let P_S, P_D, P_A denote the probability with respect to S, D, A. Let F_S, F_D, F_Adenote the Cumulative Distribution Functions (CDFs) for S, D, A with $F(x) = P(X \leq x)$.

Definition 3. An operation on margins is called Margin Distributing if it satisfies the following conditions. There exists a value θ^*, $\forall\theta < \theta^*$, $P_S(M < \theta) > P_A(M < \theta)$ until both are 0 at some θ_0; $\forall\theta > \theta^*$, $P_S(M > \theta) > P_A(M > \theta)$ until both are 0 at some θ_1.

Theorem 1 *Let D be a distribution over $X \times Y$, and let S be a sample of m examples chosen independently from D. Assume the base classifier space H has VC-dimension d, assuming $m \geq d \geq 1$. Then with probability at least $1-\delta$ over the random choice of training set S, every ensemble $f \in C$ satisfies the following bound for all $\delta > 0$:*

$$P_{\mathcal{D}}[\textit{margin}(f,x,y) \leq \psi] \leq P_S[\textit{margin}(f,x,y) \leq \theta] + O\left(\frac{1}{\sqrt{m}}\left(\frac{d\log^2(mk/d)}{\theta^2} + \log(1/\delta)\right)^{1/2}\right) \quad (2)$$

Where

$$\theta > \psi,\ C = \left\{ f : (x,y) \rightarrow \frac{1}{N}\sum_{i=1}^{N} h_i(x,y) | h_i \in \mathcal{H} \right\}.$$

Note that

$$P_{\mathcal{D}}[\text{margin}(f,x,y) \leq \psi] = EI(y(x) \neq V(h_1(x), h_2(x), ..., h_m(x)),\ \forall x \quad (3)$$

is the generalization error for ensemble f. For a proof, see Schapire et al. (1997). Theorem 1 gives the upper bound for generalization error of f in terms of margin distribution for training examples and a constant involving θ, δ, m, d, k. N is the number of classifiers in the ensemble and k is the number of classes in the classification problem. Boosting and MxBoost both aim to reduce the generalization error through reducing its upper bound, which is to reduce $P_S[\text{margin}(f,x,y) \leq \theta]$.

Remark 3: *MxBoost is approximately a Margin Distributing operation.*

This remark follows from the argument in Section 2.1. Empirical observations are shown in Figure 8 of You and Chang (2002). Note that traditional Boosting (Friedman et al., 1998; Schapire et al., 1997) is generally a Margin Distributing operation.

Remark 4: *F_S, F_A intersect at only $\theta^* \in (\theta_0, \theta_1)$, if F_S, F_A are continuous at θ^*.*

Proof: See You and Chang (2002). □

Remark 5: *MxBoost results in a tighter upper bound for the generalization error if $\theta^* > \psi$.*

Proof: When $\theta^* > \psi$, according to Remark 4 and Definition 2, for any $\psi < \theta < \theta^*$, we have that $P_S[\text{margin}(f,x,y) \leq \theta] > P_A[\text{margin}(f,x,y) \leq \theta]$. If the term $O(.)$ in Theorem 1 is insignificant, the smaller value of θ will provide tighter upper bound for the generalization error. Thus from Theorem 1, the upper bound for generalization error $P_{\mathcal{D}}[\text{margin}(f,x,y) \leq \psi]$ will decrease with a large probability. Therefore, the ensemble f is very likely to get better after MxBoost. □

3. Experimental Setup

We tested MxBoost with the intent of achieving five central goals.

1. Quantify the reduction of classification error (training and testing) on different datasets using different ensembles.
2. Compare MxBoost with Boosting.OC (Schapire, 1997) in terms of classification accuracy and classification cost.

3. Find out what effect MxBoost has on the margins.
4. Investigate whether MxBoost suffers from the overfitting problem.
5. See how different values of β can affect MxBoost.

Section 3.1 described two ensemble schemes we tested. Section 3.2 introduces the base classifier for the ensemble schemes. Section 3.3 provides the information of the datasets we used in our experiments.

3.1 Classifier Ensembles

In Section 2.1, we have described OPC. In this section, we review briefly another ensemble scheme we tested: ECOC.

Error Correcting Output Coding (ECOC) was suggested by Dietterich and Bakiri (1995) for solving k class classification problems using only binary classifiers. The steps of a typical ECOC ensemble can be described as follows:

- Produce a k by m binary coding matrix Z (i.e., a matrix of zeros and ones). Each class has a binary string of length m. We will refer to those strings as *codewords*.
- Use the first column of the coding matrix Z to create two $super$ groups by assigning all groups to super group 1 according to the first bit of their codewords. Train a binary classifier on these two super groups.
- Repeat this process for column $2, 3, ..., m$ to produce m trained classifiers.

Each classifier of ECOC provides a one-to-many confidence vote for all classes coded the same. The combined classifier, however, provides a one-to-one mapping from the original space to the K classes. Let Z_{ij} denote the jth bit of class i, $h_i(x)$ be the probability output of classifier h_i on an instance x. The L_1 distance between an example x and class j is:

$$D_H(y(x), j) = \sum_{i=1}^{m} |h_i(x) - Z_{ij}|. \quad (4)$$

The voting function is defined as:

$$H(x) = V(h_1(x), h_2(x), ..., h_m(x)) = \text{argmin}_i D_H(y(x), i).$$

The margin for ECOC is given as follows.

$$\begin{aligned} & M(y(x), h_1(x), h_2(x), ..., h_m(x)) \\ = & \min_{i \neq i^*} D_H(y(x), i) - D_H(y(x), i^*), \end{aligned}$$

where i^* is the true class for x. $\min_{i \neq i^*}$ gives the wrong class that receives the most votes among all wrong classes.

In our experiments, the coding matrices are chosen to achieve maximum row separation and column separation (see Dietterich and Bakiri (1995)).

3.2 Base Classifier

In this section, we consider Support Vector Machines (SVM) (Scholkopf et al., 1999; Smola et al., 2000; Vapnik, 1998) as our base classifiers for the following reasons. **1)** As a core machine learning technology, SVMs have strong theoretical foundations and excellent empirical successes Burges (1998); Lin et al. (2000); Smola et al. (2000); Vapnik (1998). **2)** By using various kernels, SVMs include a rich collection of classifiers. Some of them can produce only hyperplane classification boundaries, but others can produce highly complex boundaries depending on the kernel used.

In this study, we tried several kernels as base classifiers. We present the results with polynomial kernel of degree four. We also tried Gaussian kernels with different bandwidth values for which MxBoost works very similarly.

Following the research of Goh et al. (2001) and Platt (1999), we can use the sigmoid fitting to map its outputs into probabilities as follows (c is chosen empirically).

$$P(y = 1|f) = \frac{1}{1 + e^{-cf}}.$$

3.3 Datasets

For our empirical evaluation of MxBoost, we used the machine learning benchmark datasets from UCI Machine Learning Repository and datasets from the image classification domain. Seven datasets were tested. Six came from UCI Machine Learning Repository: ECOLI, GLASS, VOWEL, VEHICLE, YEAST, and SATIMAGE. One came from the image classification domain: CWLIMAGE. Table 1 summarizes datasets we used in our experiments (in order by size). Detailed descriptions of the two image datasets are given below (descriptions for other datasets can be found in Ripley (1994) and Merz and Murphy (1998)):

SATIMAGE. This dataset is taken from Merz and Murphy (1998). The goal is to identify seven soil types using 4 multi-spectral band values of pixels in 3×3 neighborhoods in a satellite image. There are 36 attributes (4 spectral bands $\times$ 9 pixels in the neighborhood). We randomly selected 2000 observations for our experiments.

CWLIMAGE. This dataset is taken from Goh et al. (2001). It consists of 1920 images collected from Corel Image CDs and the internet. They belong to fifteen categories – *architecture, bears, clouds, elephants, fabrics, fireworks, flowers, food, landscape, people, objectionable images, textures, tigers, tools* and *waves*. In this set, we extracted 108 color and 36 texture features. Eight sample tiger images are provided in Figure 4.

The experiments were carried out using 5-fold cross validation. Each dataset is split into five randomly chosen subsets

Table 1. Description of Datasets

Name	# of Classes	# of Features	Size
GLASS	6	9	214
ECOLI	8	8	336
VEHICLE	4	18	846
VOWEL	11	10	990
YEAST	8	10	1484
CWLIMAGE	15	144	1920
SATIMAGE	6	36	2000



Figure 4. Sample Tiger Images from CWLIMAGE

of equal size. In each run, one subset was set aside as the testing set and not used in training. In doing so, we avoid bias in choosing the training set and testing set.

4. Results and Discussion

Here we present our analysis of the results, organized with regard to the five central goals listed in Section 3.

4.1 Classification Error Reduction

After testing two ensembles, OPC and ECOC, on seven datasets, we report the average improvement of classification errors on both the training and testing sets. After performing MxBoost, the classification errors were measured at the iteration where the training error was minimal.

Let $Before_{CV_i}, After_{CV_i}$ be the classification errors before and after MxBoost for cross validation set i. We calculate the *Average Improvement* for five cross-validation runs as

$$\text{Average Improvement} = \frac{1}{5}\sum_{i=1}^{5}\frac{Before_{CV_i} - After_{CV_i}}{Before_{CV_i}}.$$

Table 2 shows both the training and testing improvement after 20 iterations of MxBoost on OPC and ECOC. The results on all datasets show that MxBoost is effective in reducing class-prediction errors. Dataset VOWEL (with ECOC) and VEHICLE (with OPC) enjoy the greatest reduction in testing error with 34.7% and 24.8%, respectively. Dataset YEAST is the most difficult dataset, averaging only 2.1% and 1.5% reduction in testing errors.

Table 2. Average Improvement of Classification Error After MxBoost

		Avg. Improvement	
Datasets	Type	OPC	ECOC
ECOLI	Train	15.9%	11.0%
	Test	16.8%	20.4%
GLASS	Train	20.5%	27.5%
	Test	11.8%	13.0%
VOWEL	Train	38.1%	16.2%
	Test	34.7%	12.1%
VEHICLE	Train	36.0%	38.8%
	Test	16.6%	24.8%
YEAST	Train	6.0%	0.8%
	Test	2.1%	1.5%
SATIMAGE	Train	16.7%	18.9%
	Test	11.0%	15.3%
CWLIMAGE	Train	57.4%	20.0%
	Test	7.9%	10.1%

4.2 Boosting.OC vs. MxBoost

To compare MxBoost with traditional boosting schemes, we implemented Boosting.OC and compared it to MxBoost. In Figure 5, Curve BoostOC1 shows the testing error of each iteration of Boosting.OC. We can see that without aggregating the results from all boosting iterations, the classifiers produced by individual boosting rounds perform much worse than MxBoost on all datasets. Therefore, given classification time constraint, MxBoost is a more attractive alternative.

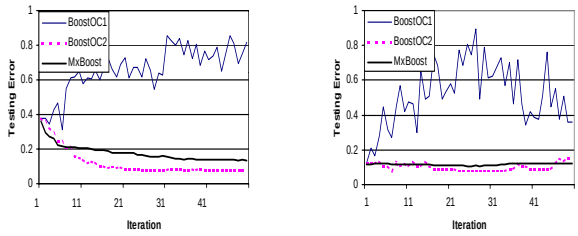
Next, we compared MxBoost with Boosting.OC when it uses classifiers produced by all boosting iterations. The classification time of Boost.OC ensemble grows linearly with respect to the number of boosting iterations. The Curve BoostOC2 shows that the testing error of the aggregated prediction of Boosting.OC marginally outperforms MxBoost on the VOWEL and GLASS datasets, and is equivalent to MxBoost on the other datasets. (Due to space limitation, we show ECOLI and VEHICLE as representatives. Results for other datasets are provided in Figure 9 of You and Chang (2002).)

4.3 Effects on the Margins

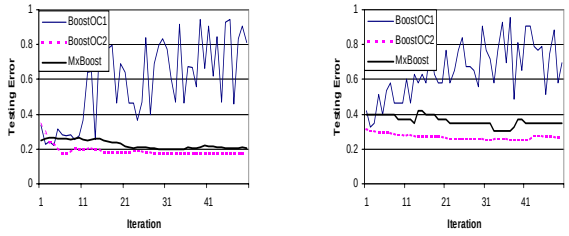
In this section, we show the effect of MxBoost on the margin of the training sets.

Figures 6 (e), (f), (g), and (h) present the boxplots of margins at different iterations using dataset VOWEL ((e), (f)) and SATIMAGE ((e), (f)). (As commonly defined, a boxplot contains a five-point summary of data, from the bottom up: Min, 1st Quartile, Median, 3rd Quartile, and Max.) Figures 6 (e) and (g) use OPC whereas (f) and (h) uses ECOC.

All the margin boxplots show that after more iterations, the



(a) VOWEL (b) ECOLI



(c) VEHICLE (d) GLASS

Figure 5. Boosting.OC vs. MxBoost

negative margins are reduced in magnitude. At the same time, positive margins are also reduced while in-between margins are increased.

4.4 Does MxBoost Overfit?

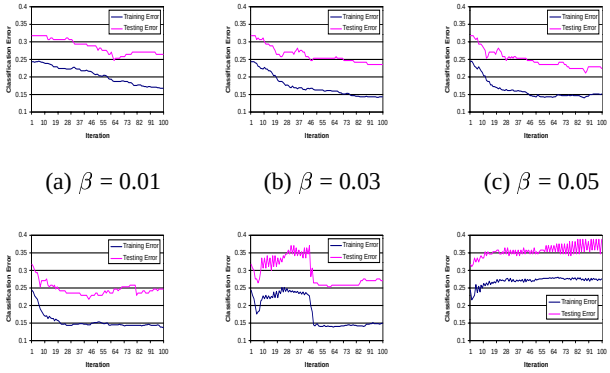
During our empirical study on overfitting, MxBoost exhibited two common patterns. The first pattern is that the testing error keeps decreasing with the training error. An example is given in Figure 6 (a) and (b) using dataset VOWEL. In Figure 6, (a) and (b) are two different runs using ensembles OPC and ECOC, respectively. In both cases, the testing error rate keeps decreasing as training error decreases. Note that $\theta^* > 0$ from iteration $1 \rightarrow 20 \rightarrow 40 \rightarrow 60 \rightarrow 80 \rightarrow 100$ (see Figure 8 in You and Chang (2002)), Remark 5 in Section 2 is thus justified. This pattern is similar to that of Boosting. But MxBoost attains the same effect without introducing new classifiers into the ensemble.

The second pattern is that the testing error goes down first, but it goes back up as soon as the training error goes up. An example is shown in Figure 6 (c) and (d) using dataset SATIMAGE. Figures 6 (c) and (d) show the classification rates for two different runs using ensembles OPC and ECOC, respectively. Note that $\theta^* > 0$ from iteration $1 \rightarrow 40$, but $\theta^* < 0$ from Iteration $40 \rightarrow 100$ (see Figure 8 in You and Chang (2002)), Remark 5 in Section 2 is also justified. Therefore both patterns can be explained by Remark 5 we proved in Section 2.

Overfitting occurs when the smallest training error corresponds to a higher testing error. In our study, MxBoost proved surprisingly resistant to the overfitting problem. Over 7 datasets $\times 5$ cross validations $\times 2$ ensembles $= 70$ runs, only 3 runs exhibit a tendency toward overfitting. In most cases, when the training error is minimized, the corresponding testing error is also minimized, or close to being minimized. MxBoost should run as long as applications allow. At the iteration in which the training error is minimized, the ensemble should be used for classification.

4.5 Sensitivity Analysis of β

As discussed in Section 2, parameter β controls how aggressively the weights are changed. If β is relatively small, the weights are changed slowly. The rate of convergence will also be slow (see Figure 7 (a) and (b)). When β is relatively large, the weights get updated too quickly, which may cause divergence (see Figure 7 (e) and (f)). A value of β that results in fast and steady convergence, based on our experiments, is between 0.03 and 0.10.



(a) β = 0.01 (b) β = 0.03 (c) β = 0.05

(d) β = 0.10 (e) β = 0.20 (f) β = 0.40

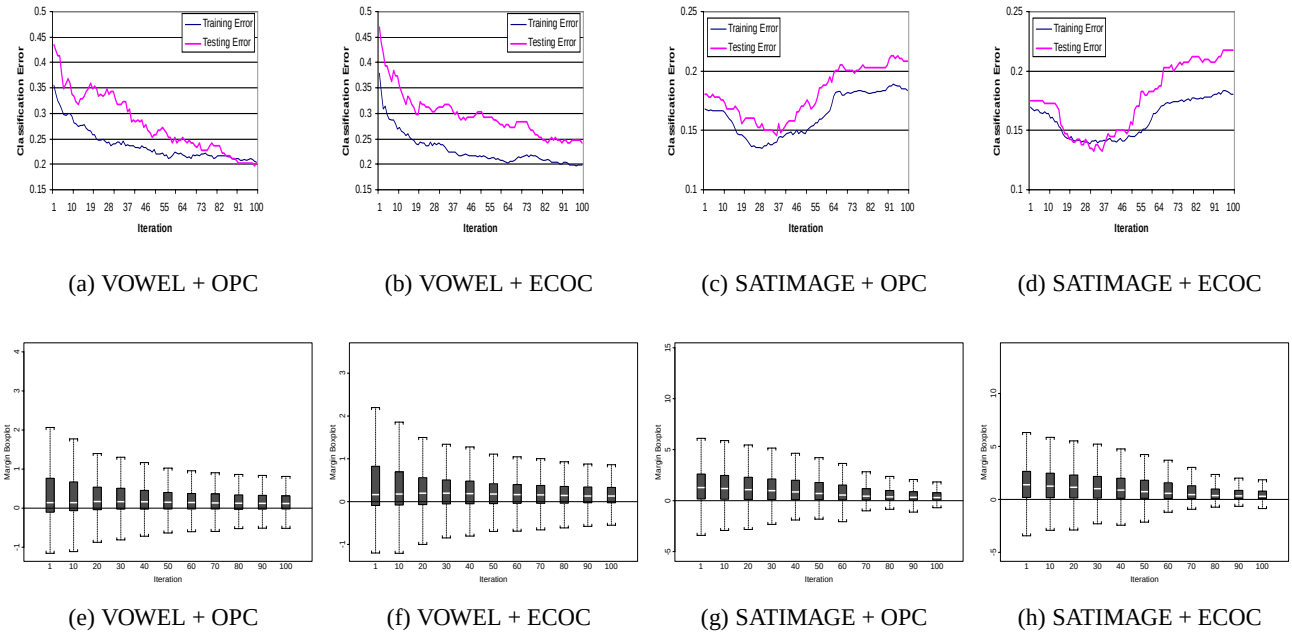
Figure 7. The Sensitivity of β: Classification Error vs. Iteration at different values of β. Dataset: VEHICLE. Ensemble: OPC.

5. Conclusion

In this paper we propose MxBoost based on the heuristics that classifiers should make mistakes mutually exclusively. We provide theoretical justifications and conduct empirical study to show that MxBoost is able to perform class prediction both quickly and accurately. Although MxBoost does not conserve training time compared to traditional boosting schemes, its classification time is substantially reduced, and hence it is an attractive scheme for online applications that require performing quick class prediction.

References

Breiman, L. (1998). Arcing classifiers. *The Annals of Statistics, 26(3)*, 801–849.



(a) VOWEL + OPC (b) VOWEL + ECOC (c) SATIMAGE + OPC (d) SATIMAGE + ECOC

(e) VOWEL + OPC (f) VOWEL + ECOC (g) SATIMAGE + OPC (h) SATIMAGE + ECOC

Figure 6. MxBoost Pattern 1: Testing error rate decreases as training error rate. Dataset used: VOWEL. Ensembles used: ECOC (left) and OPC (right)

Burges, C. (1998). A tutorial on svm for pattern recognition. *Data Mining and Knowledge Discovery 2*, 121–167.

Dietterich, T. G., & Bakiri, G. (1995). Solving multiclass learning problems via error-correcting output codes. *Journal of Artificial Intelligence Research, 2*, 263–286.

Friedman, J. (1996). Another approach to polychotomous classification. *Stanford Technical Report*.

Friedman, J., Hastie, T., & Tibshirani, R. (1998). Additive logistic regression: a statistical view of boosting. *Dept. of Statistics, Stanford, University Technical Report.*

Goh, K., Chang, E., & Cheng, T. (2001). Svm binary classifier ensembles for multi-class classification. *Proceeding of ACM Internation Conference on Information and Knowledgement Management (CIKM)* (pp. 395–402).

Lin, Y., Wahba, G., Zhang, H., & Lee, Y. (2000). *Statistical properties and adaptive tuning of support vector machines* (Technical Report). Statistics, Univ. of Wisconsin.

Merz, C., & Murphy, P. (1998). UCI Repository of machine learning databases. www.ics.uci.edu/∼mlearn/mlrepository.html.

Platt, J. (1999). Probabilistic outputs for support vector machines and comparison to regularized likelihood methods. *Advances in Kernel Methods – Support Vector Learning*. MIT Press.

Ripley, B. D. (1994). Neural networks and related methods for classification. *J. Royal Statist. Soc. (Series B)(with discussion)*.

Schapire, R. (1997). Using output codes to boost multiclass learning problems. *Proceedings of Fourteenth International Conference*.

Schapire, R. E., Freund, Y., Bartlett, P., & Lee, W. S. (1997). Boosting the margin: a new explanation for the effectiveness of voting methods. *Proc. 14th International Conference on Machine Learning* (pp. 322–330). Morgan Kaufmann.

Scholkopf, B., Burges, C., & Smola, A. (Eds.). (1999). *Advances in kernel methods – support vector learning*. Cambridge, Massachusetts: MIT Press.

Smola, A., Bartlett, P., B.Scholkopf, & Schuurmans, D. (Eds.). (2000). *Advances in large margin classifiers*. Cambridge, Massachusetts: MIT Press.

Vapnik, V. (1998). *Statistical learning theory*. New York: Wiley.

You, H., & Chang, E. (2002). *Mxboost: Mutual-exclusive boosting for online image classification* (Technical Report). Department of Electrical & Computer Engineering, Multimedia Database Lab.